AI Engineering and Claude Architect · Part I, Foundations

Working With Claude, Lesson 1

The Messages API: Stateless by Design

Your chatbot worked perfectly in testing. In production, behind three replicas, it started forgetting what customers said a moment ago. Claude had never remembered anything. Your service had been doing all the remembering, and only one replica knew.

Module	AI03 of 13
Lesson	1 of 6
Prerequisites	AI02 L1, L5, L6
Exam relevance	**Direct.** `stop_reason` (TS 1.1), passing complete history (TS 5.1), `max_tokens` and system prompts (appendix). Authentication details are out of scope
Reading time	~40 minutes

Contents

1. The Messages API: Stateless by Design

Lesson 1 of 6 · Working With Claude

The Messages API: Stateless by Design

Key idea

Every Claude feature goes through one stateless endpoint, `POST /v1/messages`: you send the complete conversation, and you get back content blocks and a `stop_reason`. Claude holds no memory between requests by default, so your application owns the conversation, and that's a design freedom as well as a duty. Two rules prevent most production bugs: append the whole response to your history, not only its text, and check `stop_reason` before you read the content.

> *Before: tokens, generation and stop reasons (AI02 L1, L5). Teaches: request and response anatomy, roles, content blocks, conversations, stop reason handling, usage, errors and retries. Used by: every later lesson. AI06 L2 builds the agentic loop directly on this, and AI08 L1 turns it into a Spring Boot service.*

The hook: the chatbot that forgets

A customer talks to your bank's assistant:

> **Customer:** My card ending 4471 was charged twice. **Assistant:** I can help. When did this happen?
> **Customer:** Yesterday. **Assistant:** I'm sorry, could you tell me what happened yesterday?

Your code keeps a `List<Message>` per session in an in-memory `HashMap`. The load balancer sent the third message to a different pod, and that pod's map was empty, so Claude received only "Yesterday."

Key idea

Claude didn't lose anything. It never had it. The Messages API keeps no conversation between plain requests. Each request has to contain everything Claude should know. Treat it like a pure function: `reply = f(system, tools, messages)`.

The Java instinct here is exactly right. You already know how to make a stateless service work behind a load balancer: move session state to an external store.

Intuition: a brilliant consultant with amnesia

Imagine a consultant who forgets everything the moment they leave the room. To get useful work, you hand them the complete case file on every visit: who the client is, what was said, and what's been tried. They read it all, give their answer, and leave. Their advice is excellent, and you're the only one who keeps the file.

That sounds wasteful. It also means **you control what's in the file.** You can remove irrelevant pages, summarize old ones and put the most important facts on top. AI03 L6 turns that into a discipline.

Anatomy of a request

```
{
  "model": "claude-opus-5-5",
  "max_tokens": 1024,
  "system": "You are a support assistant for Acme Bank.",
  "messages": [
    {"role": "user", "content": "My card ending 4471 was charged twice."},
    {"role": "assistant", "content": "I can help. When did this happen?"},
    {"role": "user", "content": "Yesterday."}
  ]
}
```

Field	Required	Purpose	Common mistake
`model`	Yes	Which model runs this	Hard-coding it in many services (AI02 L7)
`max_tokens`	Yes	An enforced ceiling on output tokens. The model isn't aware of it	Setting it low and getting truncated output
`messages`	Yes	The conversation, alternating user and assistant	Sending only the latest message
`system`	No	Instructions and context that frame the whole conversation	Putting long, changing data there, which harms caching (AI03 L5)
`tools`	No	Functions Claude may request (AI06 L3)	Offering too many
`tool_choice`	No	Whether and how tools must be used	Assuming forced values work on every model (AI03 L3)
`thinking`	No	Thinking configuration (AI03 L4)	Sending retired `budget_tokens`
`output_config`	No	`effort` and structured output format	Putting `effort` at the top level

What about the assistant turn?

You'll see `assistant` messages in history, since that's how previous replies get back to Claude. **What you can't do on current models is prefill**, meaning end your request with a partial assistant message to force how the reply begins. Prefilling returns a 400 on current models. Older tutorials often use it to force JSON, and AI03 L3 shows the replacement.

Anatomy of a response

```
{
  "id": "msg_01AbC",
  "type": "message",
  "role": "assistant",
  "model": "claude-opus-5-5",
  "content": [
    {"type": "text", "text": "I can see why that's concerning..."}
  ],
  "stop_reason": "end_turn",
  "usage": {
    "input_tokens": 41,
    "output_tokens": 87,
    "cache_read_input_tokens": 0,
    "cache_creation_input_tokens": 0
  }
}
```

Content is a list of blocks

This surprises people who expect a string. A single response may contain several blocks of different types.

Block type	Meaning	Handling
`text`	Words for the user	Display it
`tool_use`	Claude wants a tool run	Execute it, and send back a `tool_result` (AI06 L2)
`thinking`	Reasoning, possibly with empty text	Pass it back unchanged. Don't edit it
Server tool blocks	Results from web search and similar	Handle by type

So `response.content[0].text` **is a bug waiting to happen**, because the first block needn't be text. Iterate and filter by type.

The complete request lifecycle

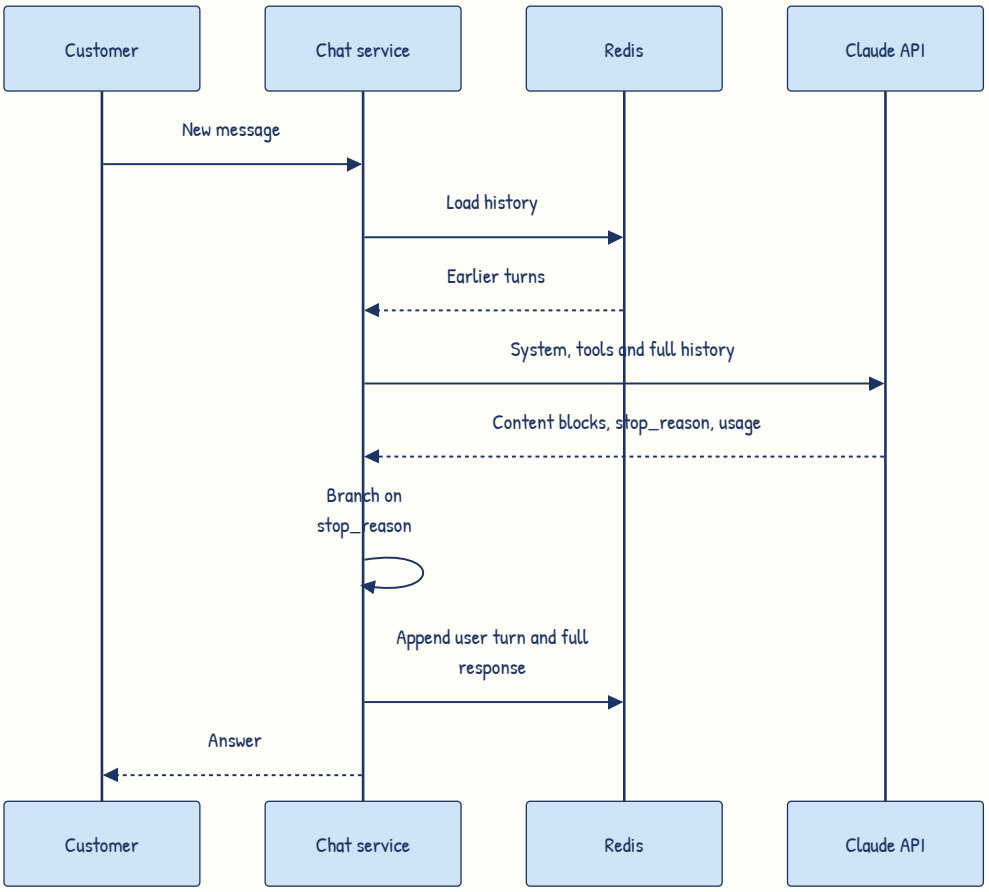


Figure 1. One turn of a conversation. The service loads history, sends it all, checks the stop reason, then saves the whole response before answering.

The order of the last steps is a design decision. Save history before you answer, so that a crash after answering doesn't leave the customer and your record disagreeing.

Rule 1: Append the whole response

Here's the Java version, from `code/ai03-claude/.../SupportChat.java`, which compiles against `anthropic-java` 3.2.0. It hasn't been run, because no API key is available.

```java
public Outcome send(String userText) {
    conversation.addUserMessage(userText);
    Message reply = client.messages().create(conversation.build());
    // Append the whole message, not just its text, so that later turns stay valid.
    conversation.addMessage(reply);
    return classify(reply);
}
```

Why does this matter? Many teams store only the visible text. That breaks in three ways:

What you dropped	What breaks
`tool_use` blocks	The next request is invalid, since each `tool_result` must match a `tool_use`
`thinking` blocks	Reasoning state is lost, or later blocks become invalid on models that check history
Compaction blocks	The API can't use its compaction state (AI03 L6)

> *Treat history as append-only*
>
> Current models add a further reason. With **preserved thinking**, on Claude Fable 5.1, Opus 5.5 and Sonnet 5.5, editing earlier turns can invalidate thinking blocks, and newer accounts get a 400 on edited history. **So if you need to trim, design it deliberately, rather than quietly rewriting old messages.** AI03 L6 covers the safe techniques.

The exam states the underlying requirement in TS 5.1: the importance of passing complete conversation history in subsequent API requests. And it tests the tool result half in TS 1.1.

Rule 2: Check `stop_reason` first

AI02 L5 introduced the values. Here's how to make the compiler help. A sealed interface forces every caller to handle each outcome:

```
public sealed interface Outcome {
    record Answer(String text) implements Outcome {
    }

    record Truncated(String partialText) implements Outcome {
    }

    record ToolRequested(Message message) implements Outcome {
    }

    record Declined(String explanation) implements Outcome {
    }

    record Unexpected(String stopReason) implements Outcome {
    }
}
```

```
static Outcome classify(Message reply) {
    StopReason reason = reply.stopReason().orElse(null);
    if (StopReason.END_TURN.equals(reason)) {
        return new Outcome.Answer(text(reply));
    }
    if (StopReason.MAX_TOKENS.equals(reason)) {
        return new Outcome.Truncated(text(reply));
    }
    if (StopReason.TOOL_USE.equals(reason)) {
        return new Outcome.ToolRequested(reply);
    }
    if (StopReason.REFUSAL.equals(reason)) {
        String why = reply.stopDetails()
                .flatMap(details -> details.explanation())
                .orElse("Declined");
        return new Outcome.Declined(why);
    }
    return new Outcome.Unexpected(String.valueOf(reason));
}
```

Why a sealed interface? A `switch` over it won't compile if a case is missing. A new `Outcome` then shows every place that needs changing. The `Unexpected` case matters too, because APIs add new stop reasons, as `pause_turn` and `refusal` show.

Reason	Exam status	Does the loop continue?
`end_turn`	Tested	No, Claude has finished
`tool_use`	Tested	Yes, after running tools
`max_tokens`	Named in the appendix	Decide: raise the limit, or continue
`pause_turn`	Current API	Yes, send the response back
`refusal`	Current API	No, handle it as a business outcome
`stop_sequence`	Current API	No

The guide focuses on `tool_use` and `end_turn`. Production code needs them all.

What usage tells you

Every response reports what it consumed.

Field	Meaning
`input_tokens`	Tokens after the last cache breakpoint
`cache_read_input_tokens`	Tokens read from cache
`cache_creation_input_tokens`	Tokens written to cache
`output_tokens`	Tokens generated, including thinking

Total input is the sum of the first three. A dashboard reading only `input_tokens` will under-report context use when caching is on, and that's a common monitoring mistake. Log all of these with a model ID and a tenant ID.

Errors and retries

HTTP	error.type	Retry?	What to do
400	`invalid_request_error`	No	Bug or unsupported parameter. Fix the request
401	`authentication_error`	No	Bad key. Alert
402	`billing_error`	No	Alert
403	`permission_error`	No	Credential lacks access
404	`not_found_error`	No	Wrong or unavailable model
413	`request_too_large`	No	Trim the request
429	`rate_limit_error`	Yes	Honor `retry-after`
500	`api_error`	Yes	Back off
529	`overloaded_error`	Yes	Back off, and maybe fall back

The SDKs retry on 408, 409, 429, 5xx and connection errors, 2 times by default. Timeouts are retried too, so **wall-clock time can reach** `timeout × (retries + 1)`. That's easy to overlook when setting your own endpoint's timeout.

Catch a chain, most specific first, so that you don't lose the difference between retryable and non-retryable errors:

```
try {
    return client.messages().create(params);
} catch (RateLimitException e) {
    throw new TryLaterException(e);
} catch (BadRequestException | UnauthorizedException e) {
    throw new ConfigurationException(e);
} catch (InternalServerException | AnthropicIoException e) {
    throw new TryLaterException(e);
} catch (AnthropicServiceException e) {
    throw new UnexpectedApiException(e);
}
```

AI10 L1 builds complete retry, circuit breaker and fallback behavior.

> ***A 200 can still be a failure***
>
> Refusals return HTTP 200 with `stop_reason: "refusal"`, and truncation returns 200 with `max_tokens`. **So a success-rate dashboard based only on HTTP status will overstate health.** Track stop reasons as a metric.

Why the obvious alternatives fall short

"Alternative"	Why it falls short
Keep history in instance memory	Fails with more than one replica, or a restart
Store only visible text	Breaks tool loops and thinking
Send only the latest message	Claude has no idea what came before
Parse replies for "I'm done"	Text isn't a control signal. Use `stop_reason` (TS 1.1)
Assume HTTP 200 means success	Hides refusals and truncation
Use `content[0].text`	The first block might not be text

How Anthropic approaches it

Anthropic's main design choice is **one endpoint with typed content blocks**. Tools and output constraints are features of that endpoint, not separate APIs. Later features were added as new block types and fields, without new endpoints.

Problem	Anthropic's answer
Long conversations resend everything	Prompt caching (AI03 L5). Beta **message threads** let the API keep a conversation briefly, so each request sends only new messages. You should still keep your own copy, because threads are best-effort
Conversations outgrow the window	Beta **compaction** and context editing (AI03 L6)
Clients and models evolve	The `anthropic-version` header pins behavior
Declines must be detectable	The `refusal` stop reason, with `stop_details`
Instructions needed mid-conversation	On supported models, a `role: "system"` message inside `messages`, which leaves the cached prefix intact

In the real world

The `anthropic-version` shown in current documentation examples is `2026-10-21`. Moving to that version is what makes sampling parameters such as `temperature` return a 400. This shows why version pinning matters: a version change can be a breaking change, so upgrade it deliberately and test it.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: history in memory. It works on a laptop, and fails as soon as you scale out or restart.

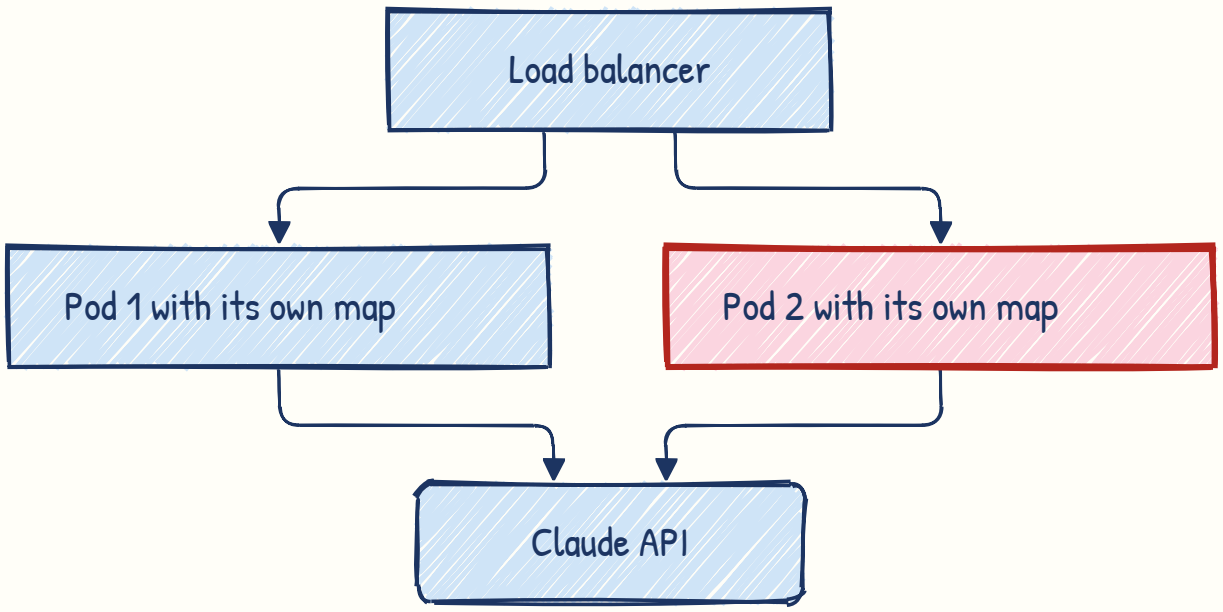


Figure 2. Baseline. Each pod holds its own conversations, so a request routed elsewhere arrives with no history.

Improved: externalize history. Any pod can serve any turn. New problem: **history now grows without limit**, so requests get slower and costlier, and eventually exceed the window. Also, two tabs writing at once can interleave turns.

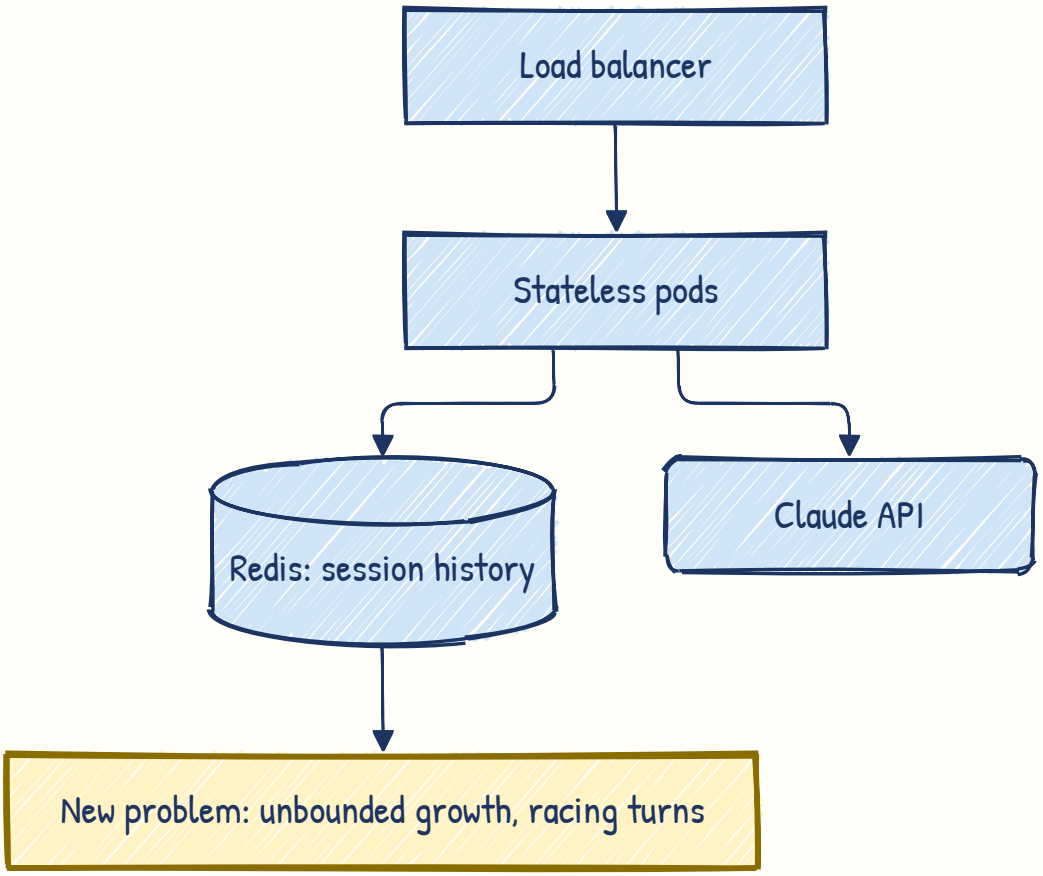


Figure 3. Improved. Pods are now interchangeable, but unbounded history and concurrent writes become the new risks.

Production: durable history and a context assembler.

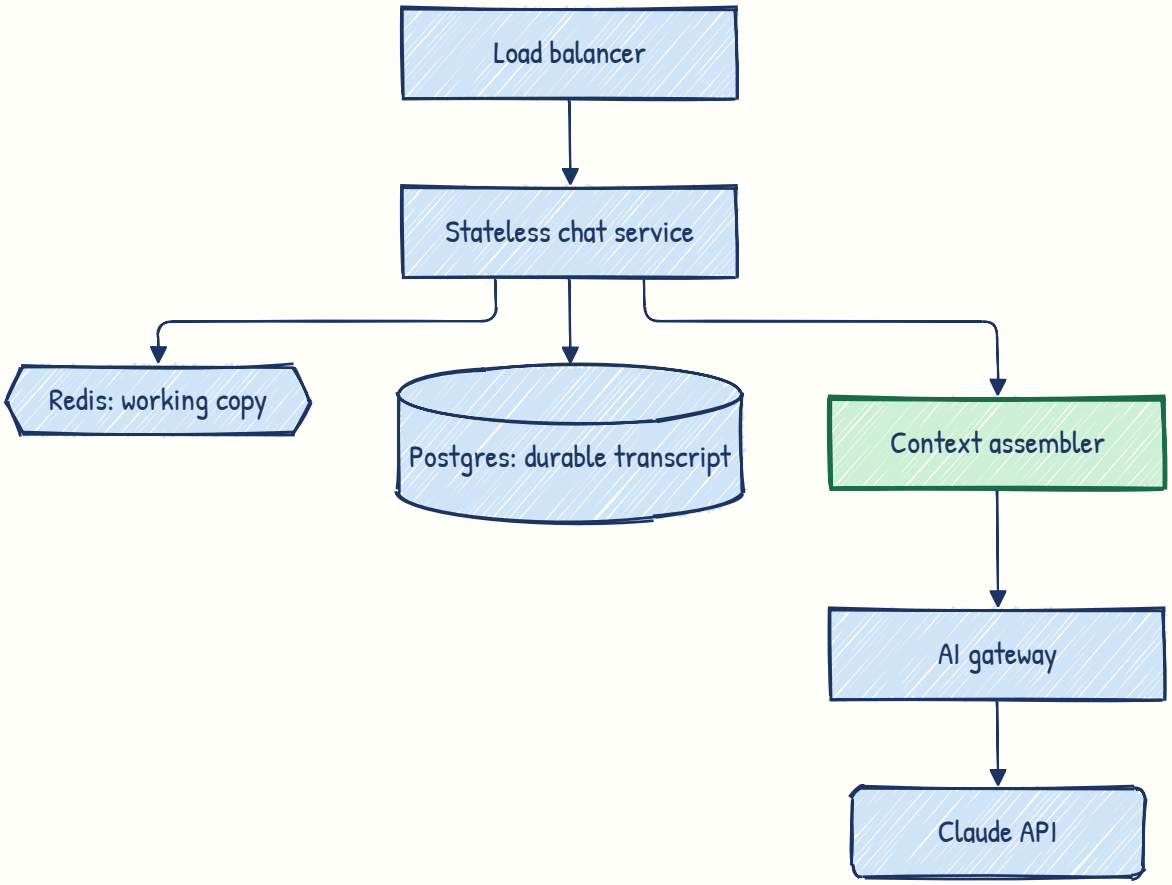


Figure 4. Production. The database is the record, Redis is a fast working copy, and an assembler decides what each request contains. A gateway centralizes retries and metrics.

Component	Why it exists	How it communicates
Stateless chat service	Scales horizontally	HTTP or SSE to clients
Redis	Fast access during an active session, with a TTL	`GET` / `SET` per session, with a per-session lock
Postgres	Durable record for audit and support	Written on each turn, possibly through an outbox (Java 22)
Context assembler	Chooses what the model sees	In-process, built in AI03 L6
AI gateway	One place for retries, routing and metrics	HTTP (AI10 L2)

Concurrency: a per-session lock or an optimistic version number stops two simultaneous turns corrupting the order. You already know this pattern from Java 15.

Failures

Failure	Effect	Containment
Redis loses a session	History gone	Rebuild it from Postgres
Crash after Claude replies, before saving	Paid reply lost, and the customer may retry	Save before answering. Use idempotency keys on any tool side effects
`max_tokens` reached	Truncated reply	Return `Truncated`, and continue or raise the limit
429 during a spike	Delayed replies	Retries, a queue, per-tenant limits (AI10 L1)
Malformed history after a bug	400 on every later turn	Validate on save. Provide a session reset

Trade-offs

Decision	Gain	Cost
Client-owned history	Full control, easy audit	You store and bound it
Beta message threads	Smaller requests	Best-effort, needs your own copy, and can't be combined with compaction
Redis plus Postgres	Fast and durable	Two systems
Sealed `Outcome` type	Compiler-checked handling	A little boilerplate
Lower `max_tokens`	Caps spend	Truncation risk. Use about 16,000 by default for non-streaming requests

When you don't need a session store: single-shot tasks, such as classifying one ticket. Each call is complete in itself.

Common mistakes

1. **Treating Claude like a session.** It has no memory of earlier requests.
2. **Saving only the text.** Save the whole `content`.
3. **Reading `content[0].text`.** Iterate over blocks by type.
4. **Ignoring `stop_reason`.** Truncation and refusals arrive as HTTP 200.
5. **Parsing natural language to end a loop.** Named as an anti-pattern in TS 1.1.
6. **Counting only `input_tokens`.** Cached tokens are reported separately.
7. **Catching one broad exception.** You lose the retry decision.

What to say in an interview

"The Messages API is stateless, so conversation memory is my application's job. I'd keep a durable transcript, plus a fast working copy, with a context assembler deciding what goes into each request. I'd append complete responses, branch on `stop_reason` before reading content, and track stop reasons as a metric, because refusals and truncation come back as a 200."

Certification lens

How this lesson maps to CCAR-F

Direct: `stop_reason` control flow (**TS 1.1**), appending results to history (**TS 1.1**), complete conversation history (**TS 5.1**). Authentication, billing and rate limits are out of scope, and they're included here for production work.

These are original practice questions, not real or leaked exam questions.

Q Q1 · Exam style, scenario: Customer Support Resolution Agent · Easy. Your agent answers the customer's second message as if the first never happened. Logs show each API request contains only the latest user message. What's the cause?

A. The system prompt doesn't instruct Claude to remember the conversation.

B. The Messages API is stateless, so the request must include the conversation history.

C. The model's context window is too small for two messages.

D. Prompt caching isn't enabled, so Claude can't retrieve earlier turns.

A **B.** Each request must contain the history (TS 5.1). A: no instruction creates memory. C: two messages easily fit. D: caching reuses a prefix you send, and doesn't store a conversation for you.

Q Q2 · Exam style, scenario: Multi-Agent Research System · Medium. Your agent loop ends whenever a response contains the phrase "research complete". Sometimes it ends early, with tool calls still pending in the same response. What's the correct termination logic?

A. Also require that the response contains no question marks.

B. Cap the loop at 10 iterations, and treat that as completion.

C. End when `stop_reason` is `end_turn`, and continue when it's `tool_use`.

D. End when the response contains any text block.

A **C.** `stop_reason` is the control signal (TS 1.1). A and D are the natural-language and text-content anti-patterns the guide names. B makes an iteration cap the primary stop. A cap can exist as a safety limit, though not as the logic.

Q Q3 · Java debugging · Medium. This works on simple chats but fails with a 400 once tools are used. Why?

```
history.add(assistantText(reply));
```

A It saves only the text, dropping `tool_use` blocks. The later `tool_result` then has no matching `tool_use`, so the request is invalid. Append the complete response, for example with `addMessage(reply)`.

Q Q4 · Sequence order · Medium. Put these in the right order for one turn: (1) branch on stop_reason, (2) load history, (3) save the user turn and full response, (4) call Claude, (5) answer the customer.

A **2, 4, 1, 3, 5.** Saving before answering means a crash can't leave the customer holding an answer your record doesn't contain.

Q Q5 · Troubleshooting · Medium. Your dashboard shows 99.9% success, but users report half-finished answers. What's likely wrong with the dashboard, and what would you add?

A It's measuring HTTP status only. A reply stopped at `max_tokens` still returns 200. Add a metric for each `stop_reason`, alert on `max_tokens` and `refusal`, and check how `max_tokens` was set.

Revision - Lesson 1

→ All features use one stateless endpoint, POST /v1/messages. Think reply = f(system, tools, messages).

→ Your application owns conversation history. Keep it outside the pod.

→ Required fields: model, max_tokens, messages. System prompt is a separate field.

→ Content is a list of typed blocks, so don't read content[0].text.

→ Append the whole response, or tool_use, thinking and compaction blocks are lost.

→ Treat history as append-only on models with preserved thinking.

→ Check stop_reason before content: end_turn, tool_use, max_tokens, pause_turn, refusal, stop_sequence.

→ Total input = input_tokens + cache_read_input_tokens + cache_creation_input_tokens.

→ 429, 500 and 529 are retryable. 400, 401, 403, 404 and 413 aren't. SDKs retry twice by default.

→ A 200 can still be truncated or refused, so track stop reasons.

→ Prefill returns a 400 on current models.

Your notes